

DeScript: A Multi-Tool Framework for Automated Unpacking and Analysis of Malicious Scripts

Author: Daniel Schalla, daniel@schalla.me

Advisor: Dr. Johannes Ullrich

Accepted: August 13, 2026

Abstract

Scripting languages are increasingly dominant malware vectors that often employ layered obfuscation techniques to slow down manual analysis by reverse engineers and to prevent pattern recognition. Existing deobfuscators are often developed for specific encoding mechanisms or programming languages, resulting in complex analysis pipelines that require multiple tools or techniques to analyze samples fully. DeScript is a multi-tool deobfuscation orchestration framework that integrates native decoders and available tools to automatically select and invoke plausible decoding techniques, including chains that span multiple languages within a single sample. This paper evaluates its effectiveness on 1,000 malicious samples (500 JS / 500 PS) and compares the results with 200 benign samples. DeScript invoked at least one tool in 75.9% of samples, with 33.9% achieving a clean terminal payload and a 1% false-positive rate for benign samples. The orchestration extends coverage beyond a single-engine tool and has shown greater effectiveness for JavaScript samples than for PowerShell samples. A majority of samples that scored zero had nonetheless been unpacked at least once, highlighting the decoupling of effective deobfuscation from the detection of malicious scripts.

1. Introduction

Over the years, malware authors and threat actors have faced increasing challenges in developing new malware capabilities due to the diversity of operating systems, patch levels, and major releases. Not only were compatibility issues a challenge, but so were increasingly advanced detection techniques that identified and prevented the execution of binary malware, either through cumbersome-to-change checksums or more advanced heuristics (AliAhmad, et al., 2023).

Script-based languages offer malware authors an attractive alternative. Because they are interpreted at runtime, they require no modification across different operating system versions and allow easy modification and restructuring to bypass detection. As a result, an ever-increasing number of malware samples in the wild have become based on scripting languages such as PowerShell and JavaScript (The Hacker News, 2025). As described in the Red Canary Threat Detection Report, PowerShell provides attackers with a versatile, flexible toolset that minimizes development costs and reduces the need for custom malware (Red Canary, 2026). Upon detecting a sample, threat actors can easily modify script structures, swap out APIs, or even dynamically generate them for the target organization with minimal overhead, thereby circumventing detection via checksums (Akamai, 2021).

The Center for Internet Security (MS-ISAC) reported in both Q3 and Q4 2024 that the JavaScript-based SocGhosh malware downloader made up 42% (CIS MS-ISAC, 2024) and 53% (CIS MS-ISAC, 2025) of CIS Top 10 malware observed, showing the prevalence of script-based malware by today's attackers. When combined with other script-based malware, such as ZPHP and LandUpdate808, the total rises to 60% of the top 10 script-based malware types observed by CIS MS-ISAC (CIS MS-ISAC, 2025).

One might initially assume that scripting languages also offer the security analyst significant advantages, given that they are essentially plain-text files that can be opened with any text editor. A sample found in the wild can be analyzed within minutes, rather than hours, compared to its binary equivalent, since no assembly reverse engineering is required. While threat actors want to leverage the advantages of scripting languages, they

actively seek to mitigate this downside through packing, encoding, and nesting. The idea was that by using techniques also used by trusted, legitimate software, such as minification within jQuery or dynamic script execution via embedded, legitimate closed-source scripts, defenders would not be able to identify them easily.

The challenge for malware analysts and reverse engineers is both the volume of new tools and the timing of their effective use. Analysts expand their toolbox step by step, largely through language-specific tools and scripts, and learn through experience when to apply different tools. As a result, the research community has continually sought to further simplify the process, often to address language-specific challenges. Herrera’s SAFE-DEOBS utilizes compiler-style static analysis to analyze and simplify obfuscated JavaScript, but noted during the analysis of a sample that resulted in obfuscated PowerShell that it “would benefit from integration with other malware analysis/deobfuscation tools” (Herrera, 2020). In contrast, Ugarte et al.’s work on PowerDrive recursively de-obfuscates PowerShell across successive layers. However, it lacks a mechanism to hand a JavaScript payload discovered in a lower layer to a JavaScript-capable tool (Ugarte, Maiorca, Cara, & Giacinto, 2019). Another approach, Li et al.’s PowerPeeler, advanced PowerShell deobfuscation through dynamic analysis, achieving high correctness by executing the script (Li, et al., 2024). Like the other mentioned tools, however, it operates solely on a single language and a single deobfuscation engine. As attackers diversify their samples across different languages, tools need to be able to respond to single samples in those languages to deliver their payloads to organizations.

This paper investigates whether an automated, multi-tool orchestration layer can recover obfuscated payloads that single-tool invocations cannot, and under what conditions it succeeds or fails. Evaluated on 1,000 real-world JavaScript and PowerShell samples, DeScript reaches a clean terminal payload for 34% of samples, 21% of all samples through at least one unpacking step, with 2 false positives on 200 benign samples, and pairs unpacking with a signal layer that flags malicious content and extracts indicators.

2. Research Method

2.1. Framework Design

The core idea behind the framework was extensibility and building on top of existing technologies, either for analysis or for unpacking. DeScript approaches deobfuscation as a recursive decision loop, in which, at each stage, available tools analyze the sample and rank it as a suitable candidate for unpacking. Each viable tool call will spawn a branch that continues to go through the above decision loop, maximizing the success rate at the cost of computation time. The analysis ranks the sample's maliciousness as it unpacks it, based on known malicious techniques or IoCs. The recursion loop will finish when either no viable tool call is identified or the configurable max-depth is reached. The extension points for `Heuristic`, `Tool`, `Sandbox`, and `Strategy` enable the framework to grow and cover additional techniques, thereby increasing its effectiveness. Figure 1 represents the high-level architecture diagram:

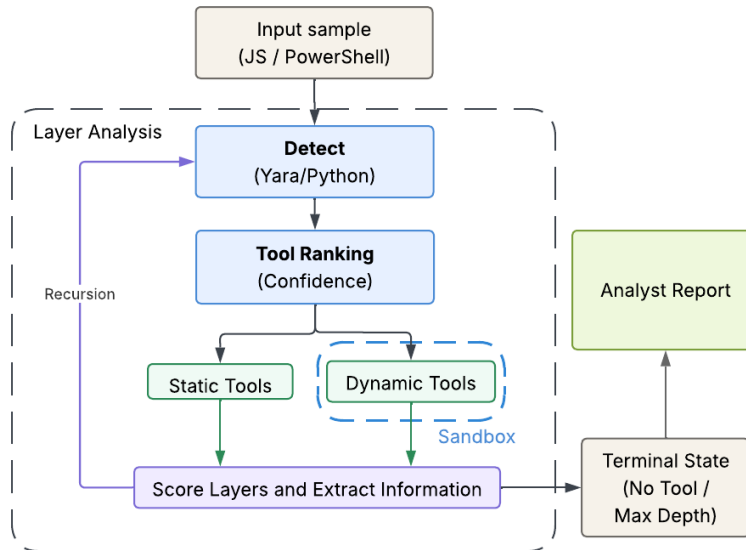


Figure 1: DeScript Framework Architecture

One key decision was to use Python as the development language, given its user base within and outside the security community. The unpacking framework supports the

integration of both native Python unpacking routines and established analysis tools, such as `xorsearch@1.11.4` (Stevens, n.d.), `box-js@1.9.27` (Muscarello, n.d.), `restringer` (HumanSecurity, n.d.), and `js-beautify@1.15.4` (Beautify Project, n.d.). The detection layer for validating unpacking success relies on both YARA rules (YARA Project, n.d.) and general behavioral heuristics, primarily to determine whether the unpacking process produced a relevant sample warranting deeper analyst review. Afterward, DeScript scores a sample by collapsing overlapping heuristics into their canonical classes, combining the author-assigned weights w_i of the fired heuristics with a noisy-OR (Pearl, 1988), flagging samples once the resulting score is ≥ 0.50 :

$$\text{score} = 1 - \prod(1 - \min(w_i, 1.0))$$

2.2. Evaluation Approach

For the evaluation, a sample set of 200 benign and 1,000 malicious scripts is used to test the developed framework. The benign samples were selected from a variety of open-source and public JavaScript and PowerShell scripts, such as jQuery, Google Tag Manager, and other commonly used files (See Appendix A). Some of those samples intentionally introduce the complexity of minification or dynamic unpacking techniques, as seen in malware, presenting both a challenge and an opportunity for the framework to demonstrate its benefits. The malicious samples were downloaded in multiple batches from MalwareBazaar (abuse.ch, n.d.) and further validated against VirusTotal (VirusTotal, n.d.). As part of evaluating the framework's effectiveness, a manual analysis using publicly available tools was conducted across multiple samples to identify limitations and scenarios in which it proves more effective than currently available platforms.

The framework was run on an isolated test VM against the previously discussed test body, allowing us to collect both analysis results and partially and fully unpacked samples. Afterward, the DeScript results are analyzed for false-positive and false-negative rates, processing time, comparison with manual analysis, activity, and effectiveness.

3. Findings & Discussion

3.1. Evaluation Setup and Metrics

The evaluation of DeScript was conducted with the engine published as commit `b518f6b8d81de8d36be63b4e4657b28cc49c4f21` and the sample corpora referenced in Appendices A and B, yielding a total of 200 benign and 1,000 malicious scripts, the latter evenly split between PowerShell and JavaScript. The malicious corpus was validated on VirusTotal, which confirmed that 994 of 1,000 samples are known to be malicious.

After running DeScript on the sample corpus, an activity rate of 75.9% was observed, indicating at least one tool invocation. 33.9% satisfied the requirement of finding a layer that is confirmed to be either a readable script or PE, with no remaining base64 encoding or high-entropy sections. The tool's effectiveness has been measured at 21.2%, measuring how many were successfully resolved with an unpacking depth of ≥ 1 . Unless otherwise stated, corpus-level metrics reported in this section were created in DeScript dynamic mode. For the analysis, the framework took 3.1h (static) and 3.7h (dynamic) under parallel execution across all samples, with 233 samples finishing in <1 s. Other samples observed an analysis timeframe beyond 1,176s, indicating both a significant sample size and complexity. A budget of 300s per sample was established and enforced only between tool calls, not during tool execution.

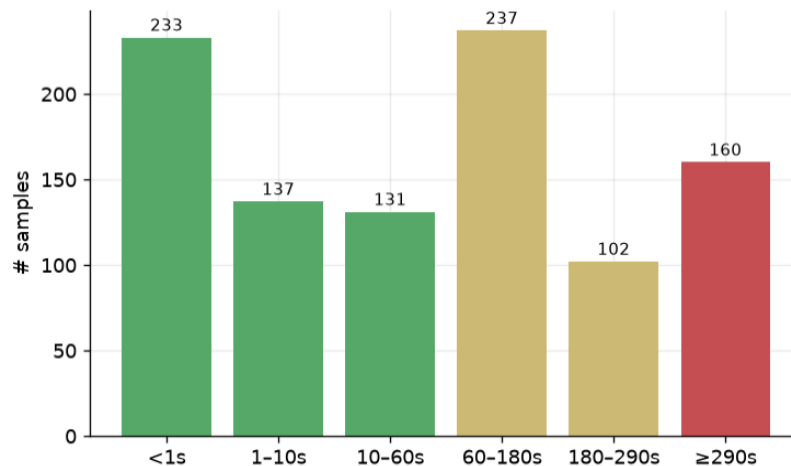


Figure 2: Dynamic Sample Analysis – Wall-clock per Sample

3.2. Deobfuscation Resolution

After analyzing the entire test corpus, at least 1 level of analysis was observed in 75.9% of the samples. Of these, at least 70.4% used a decoder such as base64, rot13, or similar, rather than clean-up techniques such as `js-beautify`.

Although 70.4% had at least one real decoder applied, only 33.9% of the samples reached a clean terminal state, defined as achieving a clean, low-entropy script or binary application. Only 21.2% of samples reached this clean terminal state after applying at least one tool call; the remainder were already clean at depth 0. Manual analysis of several partially unpacked samples confirmed successful progress through initial layers, but the framework lacked capabilities at deeper layers.

24.1% of all samples (n=241) showed no unpacking activity from the currently supported tools. As shown in Figure 4, these are predominantly PowerShell samples (196 of 241) that utilized packing methods DeScript cannot recognize or were not packed at all.

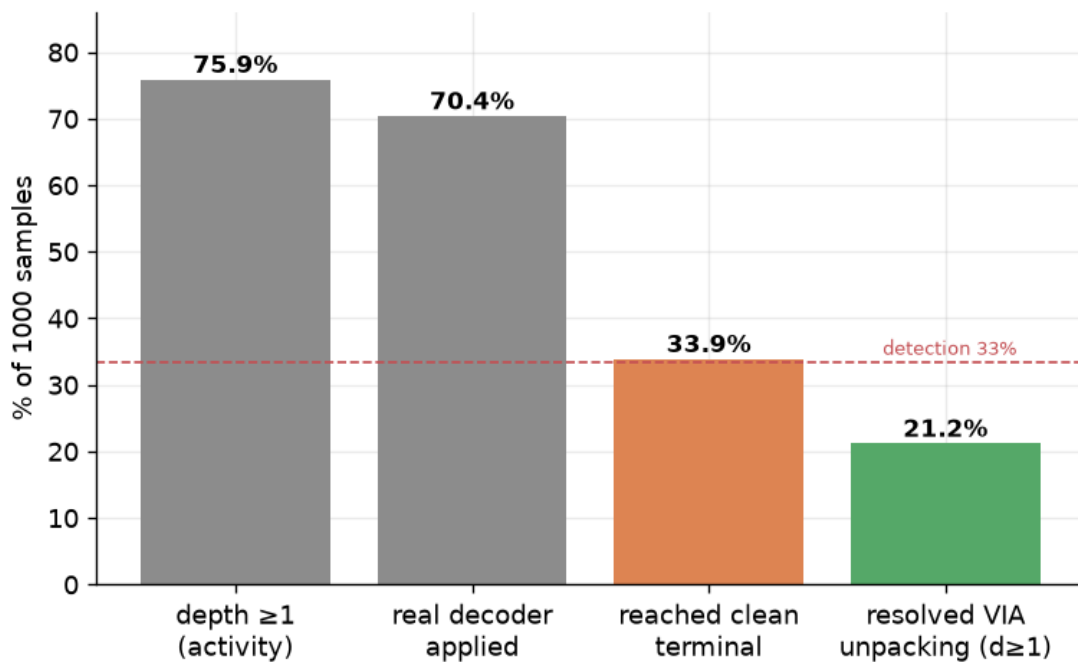


Figure 3: Resolution Funnel Across the Malicious Corpus

For the analyzed samples, JavaScript samples have, on average, a higher unpacking depth than PowerShell samples, particularly at 2-3 unpacking steps, as shown in Figure 4. This is further confirmed by the observation that 45 JS samples require 0 unpacking steps, whereas 196 PowerShell samples require 0 unpacking steps. For both PowerShell and JavaScript, samples were observed with an unpacking depth of up to 12 steps.

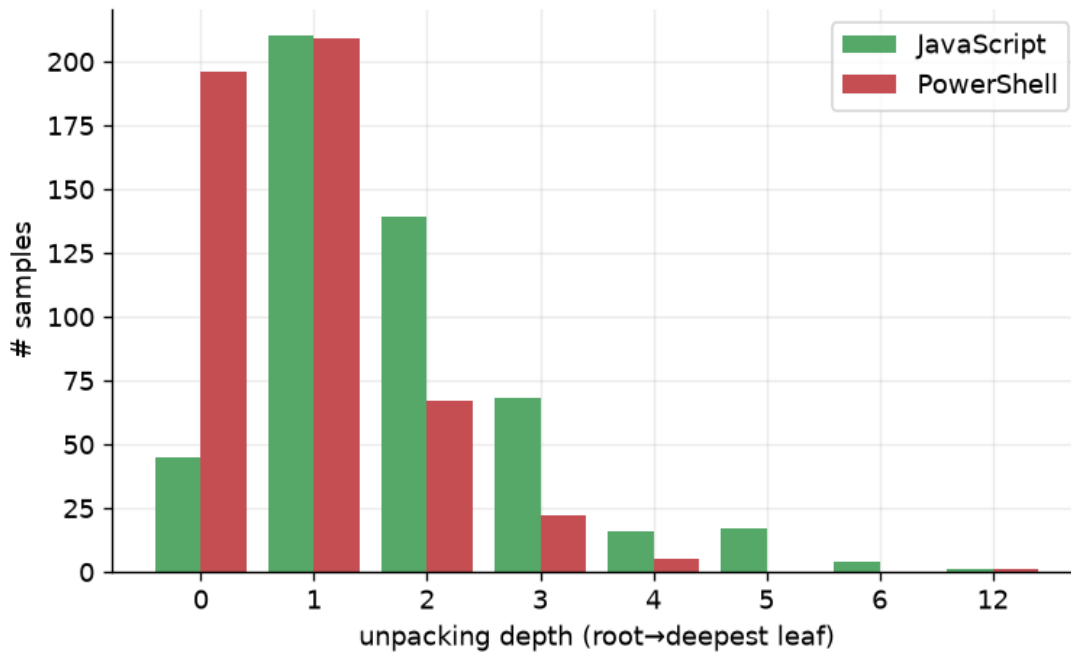


Figure 4: Unpacking Depth by Language

3.3. Orchestration Breadth

After reviewing the total number of tool invocations, the key question is whether DeScript's multi-tool capabilities are used and whether they provide benefits over a recursive iteration of a Base64 decoder.

As shown in Figure 5, generic decoders for common encoding techniques dominate the data. Base64 leads with 350 invocations. The fewest invocations are seen with ROT13 ($n=5$) and gunzip ($n=2$). For any samples reaching depth ≥ 3 ($n=121$), 82% utilize at least 2 distinct decoders. Across all samples with a depth of ≥ 2 , 52% do.

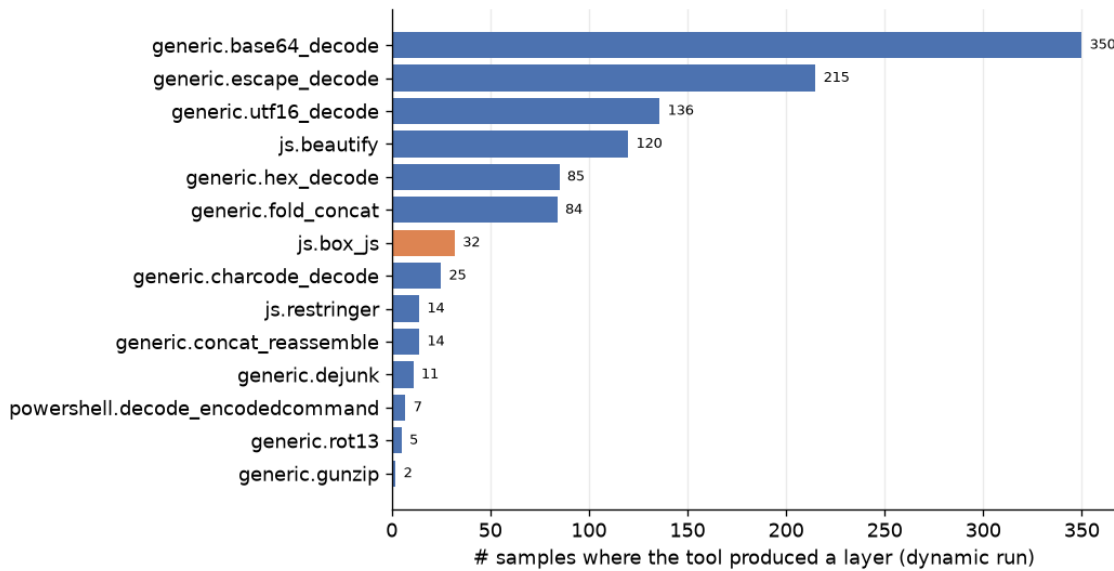


Figure 5: Tool Invocations Across the Malicious Corpus

While certain samples had a significant unpacking depth, there was no direct correlation to unpacking success or successful scoring. The sample `faf76865` had 12 successful invocations of the same base64 tool, but none reached the target state, as shown in Appendix C.

In other scenarios, such as sample `e501319e`, DeScript successfully used 5 decoders across 3 languages with a reached depth of $n=12$, achieving a clean resolved payload, as shown in Appendix D. This highlights its strength for samples that utilize different techniques across multiple languages and resolving limitations acknowledged in related work that explicitly put certain unpacking techniques out of scope: “gzip [...] is currently not supported by our system” (Ugarte, Maiorca, Cara, & Giacinto, 2019). DeScript could integrate a single-language tool, such as PowerDrive, as one step in the loop, removing that tool's limitations by handling cross-language and pre- and post-processing it cannot handle on its own.

Multiple limitations of the current implementation are evident, including stalls during the unpacking process for various reasons. For Base64 decoding, 252 samples yielded no useful output. Similarly, `xorsearch` was identified as a potential tool invocation but yielded no clean decode for 54 samples, similar to 41 samples that attempted to restructure a sample using `fromCharCode`. For 101 samples, junk removal

and Unicode interleaving failed to extract the payload. Most of the issues identified above were attributed to limitations in the current decoder implementation and content-type detection, as certain types, such as Shellcode, are not natively supported today. Enhancing said components is left to future work.

Two major influences on the tool orchestration were the orchestration strategy and the maximum node depth and number. In a prior iteration, DeScript selected only the most likely tool, potentially missing potential unpacking steps if a single sample used multiple encoding mechanisms for different parts (e.g., persistence and a dropper). The latest iteration implemented a branching strategy that defines a maximum depth of 12 for the sample run.

3.4. Detection Enablement and Indicator Recovery

To measure the effectiveness of the unpacking process, DeScript uses multiple detection capabilities to identify IoCs, suspicious API calls, and other indicators of a potentially malicious sample. The objective of this capability is to compare the scoring of depth 0 with that of lower layers, after multiple layers of unpacking have already been applied. If a sample is below 0.5 without unpacking but at or above 0.5 after unpacking, it is identified as requiring unpacking for effective categorization.

Of all 333 samples DeScript identified as malicious, 78 (23.4%) required at least one unpacking step. The distribution between the languages is relatively stark: 70 out of 180 (38.9%) JavaScript samples required unpacking, but only 8 out of 153 (5.2%) PowerShell samples did. One reason is that PowerShell samples had suspicious API invocations that could be used to identify, at an earlier stage, that the sample is likely malicious, without completing the entire unpacking process. This was further confirmed by the data presented in Section 3.2, which showed that 196 PS samples were never unpacked.

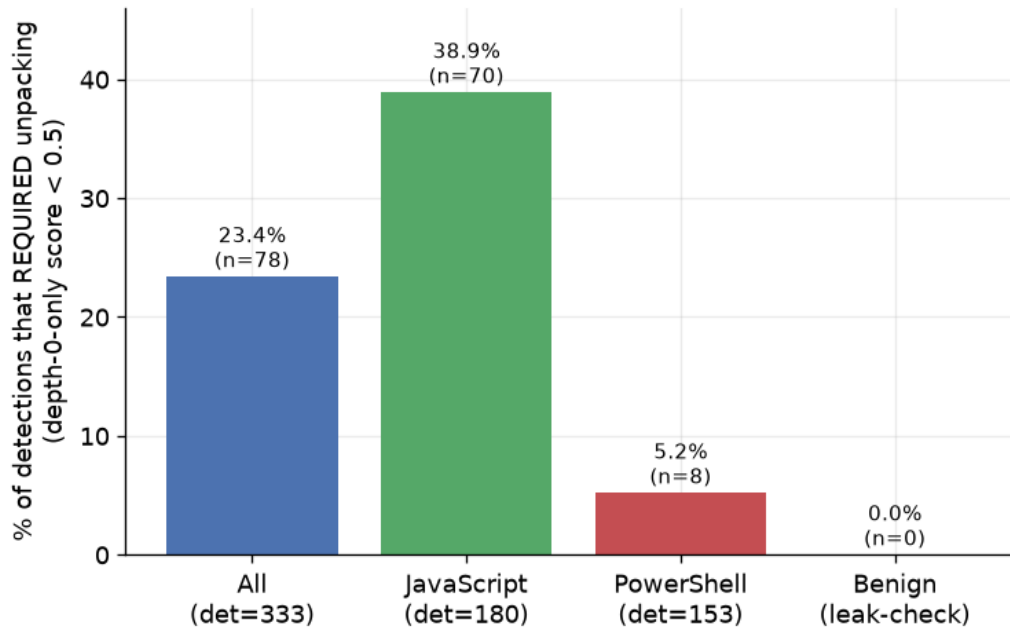


Figure 6: Detections Requiring Unpacking by Language

To review how the unpacking process enables detection, two separate samples will be investigated next. Before any unpacking steps, sample 47cae3c0 showed a score of 0.0 with no signals, consisting largely of a base64 blob:

```
# depth 0 - junk-obfuscated PowerShell
$EBhNgQ = 79 * 40 + 82
$RdpAZCZz = [System.IO.Path]::GetTempPath()
function StlutUwIVFor { return '1977' }
[...]
$data = 'CgAkAGIAeQB0AGUAcwAgAD0A[...13 KB base64...]'
[...]
([ScriptBlock]::Create((d $data))) # execute decoded payload
```

Figure 7: Payload Decoding and Execution (47cae3c0)

Analyzing the sample with DeScript recovers at depth 4 a PowerShell dropper with the analysis tree shown in Appendix E, which started to surface the actual IoCs and the behavioral scoring for download cradles and executable downloads:

```
# depth 4 - recovered download-execute cradle
[Net.ServicePointManager]::SecurityProtocol = 'Tls12'
$workDir = -join @("C", ":", "\ProgramData", "\Zooms")
$url =
"http://85[.]239[.]149[.]78:6600/ks3mscqz/dfgsdfgbvbb[.]exe"
$http = New-Object -ComObject "WinHTTP.WinHttpRequest.5.1"
$http.Open("GET", $url, $false); $http.Send()
[IO.File]::WriteAllBytes($filePath, $http.ResponseBody)
Start-Process -FilePath $filePath -WindowStyle Hidden
```

Figure 8: Download Cradle (47cae3c0)

The sample can also be observed to utilize other techniques, such as string splitting, to avoid detection from regular pattern matching. Without the prior unpacking loop, the automatic detection and categorization of the sample would not have been viable. Beyond the pure scoring, the unpacking process also enabled recovery of an IoC for the previously invisible C2 server. Across the corpus, DeScript recovered an indicator for 7.0% of samples, with trusted URLs filtered out by `_is_noise_url` (e.g., w3.org).

3.5. Decoupling Deobfuscation and Detection

Reviewing all samples that yielded a zero score in static mode shows that 359 of 498 samples successfully invoked the unpacking tool without affecting scoring. Because the samples are confirmed malicious based on both their source and VirusTotal results, this indicates a coverage gap in the detectors. However, it does not rule out the possibility of missing unpacking routines for samples that were never unpacked or were unpacked but still yielded a 0 signal, as shown by Figure 9. Both SAFE-DEOBS (Herrera, 2020) and PowerDrive (Ugarte, Maiorca, Cara, & Giacinto, 2019) explicitly described themselves as deobfuscators without integrated detection mechanisms.

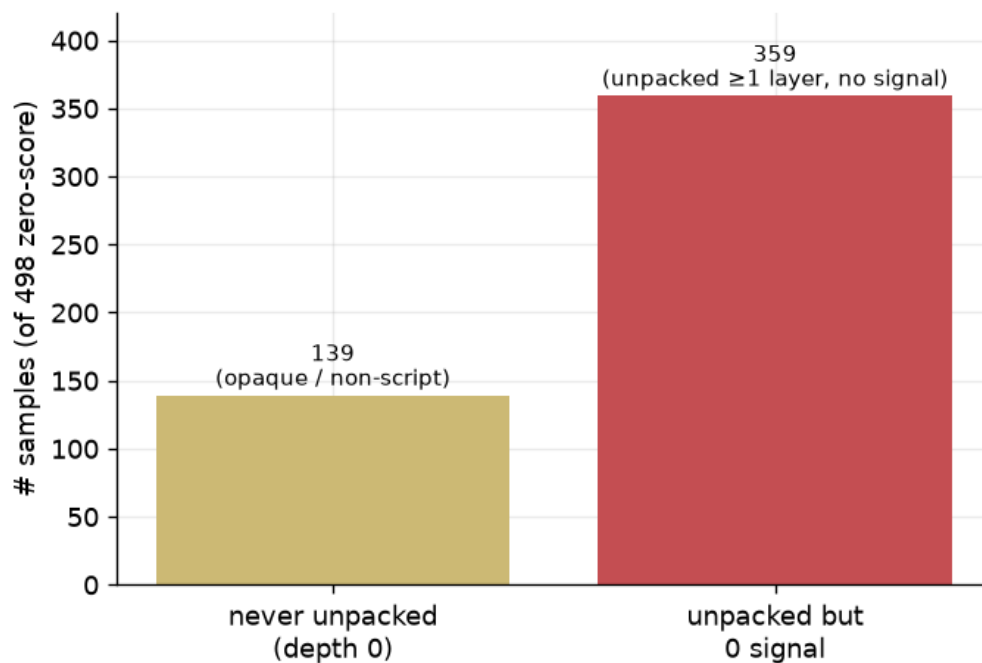


Figure 9: Activity of Zero-Score Samples

The 33% detection rate across our 1,000 samples further indicates a shortcoming in DeScript's detection mechanisms. To provide an unpacking framework, the primary angle should be deobfuscation, with detections as a secondary focus. For manual analysis, the current framework's focus on unpacking will increase success with DeScript usage.

3.6. Comparison Against Manual Analysis

To assess the effectiveness of DeScript against manual analysis, six samples have been selected from the malicious corpus for manual investigation. The selection was criteria-driven. Three samples show distinctive mechanisms in which the unpacking loop reached a clean terminal payload as defined in Section 3.2, and three samples show cases where the analysis loop failed for different reasons:

Sample	Language	Characteristics	Malware Type
S1 (e501319e)	JavaScript	Deep Nested Encoding, Multi-Language (JS, PS, VBS)	WScript Dropper
S2 (06e9c470)	JavaScript (UTF-16)	Sandbox Execution; Multi-Language (PS, JS)	Unknown (Suspected: PureLogsStealer)
S3 (2103f916)	JavaScript (UTF-16)	Stacked Static Encoding	PureLogsStealer
F1(8fa1f3e3)	JavaScript	AES-Encrypted PowerShell Stealer	Discord Token Stealer
F2(faf76865)	PowerShell	String Manipulation	KongTuke
F3(a6df7eea)	PowerShell	RC-4 Encrypted .NET Reflective Loader	AgentTesla

Table 1: Manual Sample Selection

All six samples originate from the original corpus run presented in this paper. This section aims to provide a qualitative analysis of the framework's results and highlight both successes and failures. The manual analysis was performed on a REMnux v2026.21.1 workstation (REMnux Project, n.d.) using only the included tools, such as

base64, gzip, openssl, box-js, node, and xxd. This baseline is also used to a large degree by DeScript, but in this comparison was orchestrated by a human analyst who had to sequence the integrated tools themselves.

The analysis is limited because the framework author conducted it after an automated run had completed, so it is not considered independent. This overall favors manual analysis, as certain information was known at the time of analysis. No time limit was set for sample analysis.

The analysis of the results indicates three overall outcome classes. For sample S1 (e501319e), DeScript could unpack 12 layers of a simple but nested encoding mechanism, showing the following layering and tool invocation:

d0	input .js (H3.0)	→ generic.escape_decode
d1	VBScript (H4.5)	→ generic.base64_decode
d2	VBScript (H4.0)	→ generic.utf16_decode
d3	PowerShell (H6.0)	→ generic.base64_decode
d4	binary (H7.8)	→ generic.gunzip
d5	decoded utf-16 (H4.4)	→ powershell.decode_encodedcommand
d6	PowerShell (H6.0)	→ generic.base64_decode
d7	binary (H7.7)	→ generic.gunzip
d8	decoded utf-16 (H4.4)	→ powershell.decode_encodedcommand
...	(base64 → gzip → ps-enc cycle repeats, payload shrinking each round)	
d12	PowerShell (276 B)	→ max_depth (loop cap reached)

Figure 10: S1 Analysis Tree

While the sample is easy to analyze manually, chaining multiple encoding methods makes decoding labor-intensive compared to invoking DeScript. At the same time, it highlights one limitation of the configurable `max_depth`: it is reached at layer 12. Manual analysis could fully recover the second payload file location, which would have

been recovered by layer 14 by DeScript. The manual analysis took 12 minutes, compared to 41 seconds for DeScript, showing a clear timing advantage for DeScript.

For other samples, the results are unclear. For samples S2 (06e9c470) and S3 (2103f916), DeScript effectively unpacked multiple stages to depths 3 and 5, respectively. However, manual analysis recovered additional evidence by applying currently unsupported analysis mechanisms, such as a subtraction cipher. Manual analysis revealed the final PowerShell loader for a .NET PE binary. For those samples, DeScript accelerated the analysis by resolving rudimentary steps, resulting in an overall speedup. It was not able to fully resolve the final payload in its current form. However, it collected sufficient evidence that the sample is likely malicious and showed clear signs of obfuscation. S2 and S3 were analyzed in 33.3s and 60.3s, respectively, by DeScript.

Lastly, F1(8fa1f3e3), F2(faf76865), and F3(a6df7eea) are the sample categories for which DeScript achieved very limited or no results. For sample F2, the limitations of static analysis were quickly revealed by a custom string builder that executed 2077 Remove, 1766 Insert, and 573 Replace statements. Even though the sample was emulated on the REMnux VM using pwsh, a recovery was not possible, which is currently suspected to be due to a limitation of the .NET Core APIs. For samples F1 and F3, limitations were reached early due to the lack of an AES and RC4 decryption tool. Compared to samples S2 and S3, DeScript did not provide a partial benefit here, as the missing tool calls were reached early, requiring a fallback to manual analysis.

Overall, the results indicate that DeScript accelerates sample analysis when suitable tools are available for all or earlier stages of the unpacking process. However, it is unsuitable for samples that use any form of encryption or PowerShell emulation until support for those tools is added in a future release.

3.7. False-Positive Analysis

Many techniques observed in malware also have legitimate use cases in both open-source and commercial software applications and libraries. The most prevalent motivation is protecting intellectual property for commercial organizations that publish JavaScript SDKs and want to prevent interested parties from easily understanding their inner workings. However, security through obscurity can also be a motivating factor.

To minimize the number of false positives identified by DeScript that implement the same techniques, it was decided that general obfuscation signals, such as Base64 encoding, are low-weight signals. Instead, explicit YARA rules and string detections are used to identify known malicious or at least suspicious behavior in samples.

During the framework's development phase, multiple rules initially fired on the known benign samples in the set (see Appendix A), requiring continuous tuning as the body evolved. The rule `js_eval_packer` had the goal to identify `eval()` and `Function()` invocations that are often used first to unpack malicious commands and afterward execute, but were also found in samples such as `lodash`, which, as a result, had a malicious score of 0.3 already, with samples above 0.5 being marked as potentially malicious. Since it is still a valuable signal, it was reduced from 0.3 to 0.1 to increase the available margin. The rule `decode_encodedcommand` fired on a minified jQuery instance since the regular expression was overly aggressive and misclassified regular code as base64-encoded commands. The proper optimization here was to check whether the suspected payload is a valid Base64 string and ensure it has a minimum length. These weights were tuned manually against the evaluation corpus to minimize false positives. The resulting score ranks samples for analyst triage rather than estimating a probability of maliciousness.

For indicators of compromise, many samples contained, at layer 0, a variety of benign and known domains that had to be filtered out, such as “w3.org” SVG namespaces and other well-known trusted resources that can appear in any image or XML. While not adding to the malicious signal of a sample, it causes unnecessary noise and confusion for

the analyst. This will need continuous maintenance and growth to keep filtering out known URLs that often appear in legitimate samples.

	DeScript: Benign	DeScript: Malicious
Benign (N=200)	198 (99% Specificity)	2 (1% FP Rate)
Malicious (N=1000)	667 (66.7% Miss)	333 (33.3% Recall)

Table 2: False Positive Analysis

Lastly, a major complexity comes from cradle installers such as Chocolatey, Scoop, and others that rely on `IEX` with `DownloadString`, which “is also popular in non-malicious contexts, making false positives based on this indicator a significant challenge” (Bohannon & Holmes, 2017). The same behavior is used by malware to download the second stage from the internet. Unless samples are allowed to be listed based on known benign SHA hashes, they remain marked as more malicious. The current approach is to tune the engine to barely meet the malicious marking criteria, though it may require further tuning.

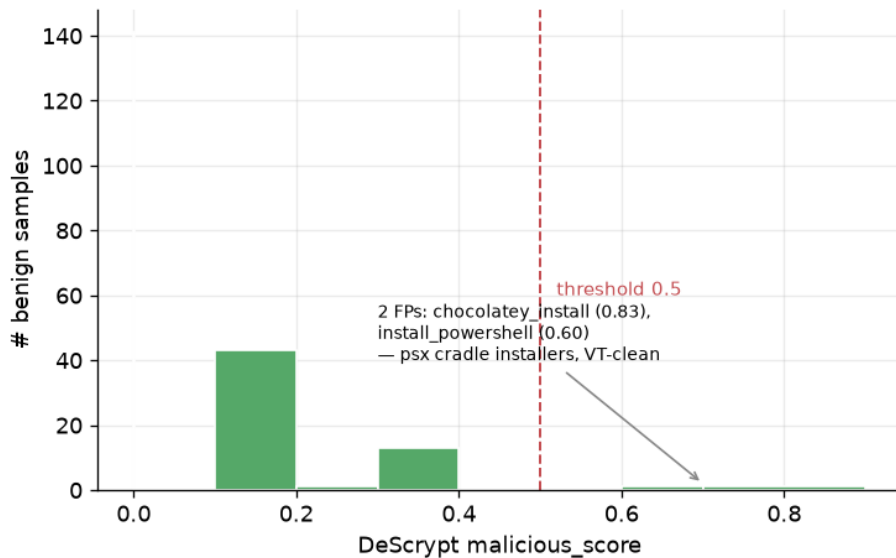


Figure 11: Benign Sample Score Distribution

The tunings above result in the benign sample body having a 1% false positive rate across clean source, minified, cradle installers, and slightly packed benign samples in our test body. New approaches and/or samples will likely again trigger a false positive over time. The benign samples are continuously evaluated as part of the framework's unit tests, ensuring that regressions are caught early and remedied.

As shown in Figure 11, many benign samples still have scores above 0.0 due to factors such as obfuscation. The deduplication based on the behavior classes described in Section 2.1, combined with score tuning, kept the overall false-positive rate within a reasonable range.

3.8. Synthesis

After reviewing the research project's findings, the data provides evidence that multi-tool orchestration can recover payloads within samples that single-tool invocations cannot, particularly for samples that harness multiple languages in their payload delivery. 21.2% of the samples yielded a clean payload, as shown in Section 3.2. Section 3.3 shows that while Base64 Encoding remains the most frequently used encoding method, other techniques are also commonly observed. The value of unpacking was also shown to be language-dependent. Unpacking enabled 38.9% of JavaScript detections that raw inspection could not reach, compared with only 5.2% for PowerShell (Section 3.4).

The manual comparison in Section 3.6 showed clear advantages in reducing manual labor and decision-making for analysts analyzing nested samples across different languages, effectively recovering lower layers and indicators of compromise that would otherwise be hidden in clear text.

Section 3.5 has established that deobfuscation and detection functionalities are decoupled. 66.7% of all malicious samples scored below the threshold, whereas the false-positive rate for benign samples is low at 1% (2 of 200).

Overall, the results indicate that DeScript is an effective triage and analysis tool for reducing initial obfuscation layers and recovering interesting indicators of

compromise. However, in its current form, it does not provide reliable detection algorithms for classifying scripts as malicious or benign.

4. Recommendations for Future Research

After reviewing and discussing the findings in the previous section, this section examines how usable the research and development is for practitioners today and what future research could follow.

4.1. Readiness Analysis and Viability for Practitioners

After reviewing the results from the initial framework on a realistic sample set, DeScript is an overall useful but still early tool for malware analysts. Its primary benefit is the rapid, low-effort triage of large sample sets by integrating it into an analysis pipeline, in which automatic layer unpacking occurred for 75.9% of samples based on available data.

In its default static mode, DeScript's decoder uses only analysis techniques that do not execute any malicious code, a choice that has proven effective across a variety of encoding methods. That said, the DeScript framework provides fairly limited value for any samples that use custom cryptography, customized packing methods, or other approaches that the framework does not support today (See Section 3.3). In static mode, the tool fails to identify the pattern correctly and therefore cannot analyze it. When executing in the supported dynamic mode with a sandbox and platforms such as `box-js`, this weakness can be partially mitigated by dynamically unpacking samples, but this comes at the cost of increased risk and environmental complexity.

The recommendation for operators is to use the tool for an effective first pass on samples to identify techniques quickly and, if successful, gain access to deeper layers, but not as a replacement for manual analysis at this stage. With proper isolation available, dynamic mode extends coverage to samples that materialize their payload only at runtime, which is most valuable for JavaScript (Section 3.4). As `box-js` is currently the only supported dynamic tool, it would need to be further analyzed to determine whether a

dynamic PowerShell tool would offer comparable efficiency and reporting gains, which would be a valuable investment for future research.

4.2. Future Research Opportunities

The current evaluation identified multiple gaps in the research project at this stage that could enable future expansion and improvement. The main areas are classifications, techniques, and accessibility.

In Section 3.7, False-Positive Analysis, it was briefly noted that many optimizations were performed manually to avoid incorrectly classifying benign samples as malicious, or vice versa. As the sample data evolves, it is likely to require a carefully evaluated approach to ensure the weights are optimized. The same can be said of the actual harness configuration, with its branching strategies and static vs. dynamic analysis, which could lead to significantly different results. As the framework matures and more test data becomes available, it will be an effective lever for increasing efficiency.

Beyond tuning the classification, the available tools have to be expanded. For samples that could not be unpacked statically or dynamically, they should be manually analyzed to determine whether the framework can be expanded with new tools, such as a native decrypter or `fromCharCode` decoders. Identifying the tools that will provide the broadest benefit will be crucial, given that not all techniques can be implemented in the near future. The framework limited initial research to PowerShell- and JavaScript-based samples but would benefit from expansion into new delivery formats, such as maldocs or other scripting languages. This would prevent the manual labor of first extracting scripts from other delivery classes and further simplify the analysis process. The same applies to dynamic analysis tools: the framework currently supports only `box-js`, which simulates a specific JavaScript environment but does not support other execution contexts, such as Node.js.

Given all of the above, it must be carefully considered which additions provide broader benefits and which are just one-time optimizations unlikely to be seen in other samples. Every added work item must be maintained over time, so overall the benefit should outweigh the maintenance cost.

5. Conclusion

With the rising prevalence and increasing complexity of script malware, this paper analyzed whether the multi-stage, multi-language deobfuscation of DeScript provides a benefit over manual, single-tool-based unpacking techniques. DeScript provided direct benefits for samples that either use multiple iterations of the same encoding technique or mix languages and encoding methods. DeScript removed at least one layer in 75.9% of samples and produced a clean payload in 33.9% of samples; 21.2% required at least one unpacking step, and 12.7% were already clean at depth 0. The orchestration of multiple tools was demonstrated in practice rather than merely being available: 82% of samples that reached an unpacking depth of three or more (n=121) composed at least two distinct decoders. This was demonstrated across multilingual samples, including unpacking a depth-12 JavaScript sample into a PowerShell payload.

First, the value of automated unpacking was highly language-dependent: 38.9% of JavaScript detections were enabled by deobfuscation, compared to only 5.2% for PowerShell samples. As highlighted in Section 4, it needs to be further analyzed whether this is a characteristic of the distributed PowerShell samples or a limitation of the currently supported decoding mechanisms. Second, the evaluation showed that deobfuscation and detection are decoupled capabilities. The majority of samples without any detection signal had nonetheless been unpacked at least once. The detection layer achieved only a 33.3% overall success rate despite successful deobfuscation passes, and improving classification therefore requires investment separate from unpacking coverage.

Overall, DeScript is best positioned as a first-pass triage tool to accelerate the investigation of unknown samples, achieving a 1% false-positive rate across 200 benign samples. The framework significantly reduces manual effort when unpacking nested, multilingual samples, especially for new and inexperienced analysts. The identified gaps consist of unsupported unpacking techniques and detection weights that require further tuning. In its current form, it complements rather than replaces manual investigation. At the same time, it demonstrates that automated multi-tool orchestration measurably reduces the effort required to unpack nested, multi-language malware.

References

- abuse.ch. (n.d.). *MalwareBazaar*. Retrieved from abuse.ch: <https://bazaar.abuse.ch/>
- Akamai. (2021, October 19). *Over 25% of Malicious JavaScript Is Being Obfuscated*. Retrieved from Akamai: <https://www.akamai.com/blog/security/over-25-percent-of-malicious-javascript-is-being-obfuscated>
- AliAhmad, A., Eleyan, D., Eleyan, A., Bejaoui, T., Zolkipli, M., & Al-Khalidi, M. (2023). Malware Detection Issues, Future Trends and Challenges: A Survey. *2023 International Symposium on Networks, Computers and Communications (ISNCC)*, (pp. 1-6).
- Beautify Project. (n.d.). *beautifier/js-beautify*. Retrieved from GitHub: <https://github.com/beautifier/js-beautify>
- Bohannon, D., & Holmes, L. (2017). Revoke-Obfuscation: PowerShell Obfuscation Detection Using Science. *BlackHat USA*.
- CIS MS-ISAC. (2024, October 23). *Top 10 Malware Q3 2024*. Retrieved from CIS Center for Internet Security: <https://www.cisecurity.org/insights/blog/top-10-malware-q3-2024>
- CIS MS-ISAC. (2025, January 31). *Top 10 Malware Q4 2024*. Retrieved from CIS Center for Internet Security: <https://www.cisecurity.org/insights/blog/top-10-malware-q4-2024>
- Herrera, A. (2020). Optimizing Away JavaScript Obfuscation. *IEEE Working Conference on Source Code Analysis and Manipulation*.
- HumanSecurity. (n.d.). *HumanSecurity/restringer*. Retrieved from GitHub: <https://github.com/HumanSecurity/restringer>
- Li, R., Zhang, C., Chai, H., Ying, L., Duan, H., & Tao, J. (2024). PowerPeeler: A Precise and General Dynamic Deobfuscation Method for PowerShell Scripts. *ACM SIGSAC Conference on Computer and Communications Security*.
- Muscarello, G. (n.d.). *CapacitorSet/box-js*. Retrieved from GitHub: <https://github.com/capacitorset/box-js>
- Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann.

- Red Canary. (2026). *Threat Detection Report*. Retrieved from Red Canary:
<https://redcanary.com/threat-detection-report/techniques/powershell/>
- REMnux Project. (n.d.). *REMnux: A Linux Toolkit for malware Analysis*. Retrieved from
REMnux Project: <https://remnux.org/>
- Stevens, D. (n.d.). *xorsearch*. Retrieved from Didier Stevens - Personal Blog:
<https://blog.didierstevens.com/programs/xorsearch/>
- The Hacker News. (2025, June 13). *Over 269,000 Websites Infected with JSFireTruck
JavaScript Malware in One Month*. Retrieved from The Hacker News:
<https://thehackernews.com/2025/06/over-269000-websites-infected-with.html>
- Ugarte, D., Maiorca, D., Cara, F., & Giacinto, G. (2019). PowerDrive: Accurate De-
Obfuscation and Analysis of PowerShell Malware. *Detection of Intrusions and
Malware, and Vulnerability Assessment, 16th International Conference*.
- VirusTotal. (n.d.). *VirusTotal*. Retrieved from VirusTotal: <https://www.virustotal.com/>
- YARA Project. (n.d.). *Writing Yara Rules*. Retrieved from YARA Documentation:
<https://yara.readthedocs.io/en/stable/index.html>

Appendix A: Benign Sample Corpus

Name	SHA256
ace-builds@1.32.7	4e0c6dffb6ee6e897100b445a01a1f83aeb7e f484144e6f4cf196898f7c675bb
alpinejs@3.13.5	ca057831ef9be3d8ee47e14078089fd2381dc 7820b4bb7fbdb85a490f5b8f68c
alpinejs@3.13.5 (src)	379399369e3b6bbe54bcff01704e4ac86add ea443557e9850e41b7b4bb4ffc4
angular@1.8.3	396dc1a03d6cc02e9c51a80246e0db53c5c8d f9bd07287e3b51bce4a29dab355
animejs@3.2.2	b5ce1be3c3f530f192e0f2571d1942846096d 66119cbada34bfdc912c4873f35
apexcharts@3.45.2	49090a28e8803db116aef2109e83e70225081 624038933cabb6d66626998d46f
axios@1.7.2	8ad11c4cb079bba93156727270f510292eedc c0716c6f21725074a59ec8b9b96
axios@1.7.2 (src)	c52af035caee76a6320e13ca3b7397b49b3e7 c19dd2d3464052702a5395afc98
backbone@1.5.0	be241a24cafe282578f2c401f940531631d50 f3d0432c43b7adc76eca67a8898
bootstrap@4.6.2	19126b874a32753d42c12dfa6c17892bfd938 20a5a5100ba1b34da4d07599b49
bootstrap@5.3.3	0833b2e9c3a26c258476c46266e6877fc7521 8625162e0460be9a3a098a61c6c
bootstrap@5.3.3 (src)	9a4a11a15db88d5fab08f59c1c34796b03f1f 15bb3cc928dd226e1c59f7f59a3
chart.js@4.4.1	d2af8974e95271638772e9e9524db5b9a6f58 d6ec2d5d781400447b4a31c681e
chart.js@4.4.1 (src)	74401d738dd3e03ee5dfb3b6841210fe2c4ea d8a960c4011ca4ba0b78a9fd8f3
cleave.js@1.6.0	7eb194c2648de022cb8f29399b9f4409d5ec0 cc5314d6e4eea175c78d1d5089a
codemirror@5.65.16	0644e6418c4129c9584b9b772b8b8bdb0b4fb ab1efd848a98baf1fe147381f22
core-js-bundle@3.36.0	93ab2f1bd5d8715376a2700cbb1523af0464e d0bcd7e54016a9589d2e81bc81d

Name	SHA256
crypto-js@4.2.0	33774f8a5df2f0cebb375107a18ed357958879bbfe35c807e27f4492a6b0720f
d3@7.9.0	f2094bbf6141b359722c4fe454eb6c4b0f0e42cc10cc7af921fc158fceb86539
d3@7.9.0 (src)	0c0d58be2106c14e242fc75bc09a17467800630751580665f6e9552a0c064ea2
datatables.net@2.0.2	263d391476883561db6e2531683a6e4298efe50674c10d11a5bb9dd4500c74d8
dayjs@1.11.10	9cfdb93f38afc2d076abecd66d32bfd3383cdf1967654ebc26a26605daf4173
dompurify@3.0.9	a3876688dffda0991175e82ff447dba95895bcc702f5388d04ffaa6240f43ab2
echarts@5.5.0	42f8329d989b6f6539dd2b15bbdf0d82025762ac112fbb60dc57b27d7bcf3946
fabric@5.3.0	48f8f0915beb512fc7fe1b48c0eeb075dd0803c790efdfd75cd007f9c3eea2ca
fb_pixel	ad5b6c8adbac6d78c46bc8037ad3961e92be9a8f0f1e4c1e15d2f43356fe88fe
flatpickr@4.6.13	1eeab1cb779471a0b0aaa93dd91c2eb1aa537d696f01ab05ea9dabc55e8525a1
fullcalendar@6.1.11	8b8bc35bd120b5a26679ab830f29ad345da898f6757c22a97f8c35801954d481
gsap@3.12.5	28033e449a31ebcc396e5be8b13b63152bf03094288fb5867034321927bce087
handlebars@4.7.8	0e5416f145e7bf16c58504356c732fe7e99671f4696194c5b140a252db02f0af
handlebars@4.7.8 (src)	9609481aa437379b76df93838cd43066db8972ba2f2efb93ed47a54e5c56a4b4
he@1.2.0 (src)	76c554d5bbfd032fe620595076a50abea5124b9cbd4e9ffe6ac94a4f855aeceb
highlightjs_11_9	837a6fa5b0c736b52bbde2b2b6190f305da3fc9ed41681db5321507057b5c846
hls.js@1.5.7	a78b3603d762428caba2ef21674e4d47fbc4e7496292b28f845ba7372849360b
htmx.org@1.9.10	b3bdcf5c741897a53648b1207fff0469a0d61901429ba1f6e88f98ebd84e669e

Name	SHA256
hubspot_forms	0613de131d163dcc3fd3b1b29a99138369d0bd922ef33e33547637342e6cdd06
immutable@4.3.5	46ad00c47cfc112ef48617ef9933a6e47c33fcd7b2b462506b46cc78a3d9c3c8
inputmask@5.0.9	0ba6c23e11cd5cb9afd3f22a1f6a9b44e3281573d1856b5c312c646204376b72
jquery@3.7.1	fc9a93dd241f6b045cbff0481cf4e1901becd0e12fb45166a8f17f95823f0b1a
jquery@3.7.1 (src)	78a85aca2f0b110c29e0d2b137e09f0a1fb7a8e554b499f740d6744dc8962cfe
jqueryui_1_13_2	9528ca634fecad433d044ddd3e6f9ce1f068d5d932dafdbb19d8e6daea1968bd
jszip@3.10.1	acc7e41455a80765b5fd9c7ee1b8078a6d160bbbca455aeae854de65c947d59e
katex@0.16.9	dc84b296ec3e884de093158f760fd9d45b6c7abe58b5381557f4e138f46a58ae
knockout@3.5.1	e8957bb182a5047b07bea0a49fd22b11614b1ab9ac5b8286fcb2259f48658e73
konva@9.3.6	7fb114bc4ce6626366cc1e2ca091275079ac70ab6ce652105e5a42a5923e1b95
leaflet@1.9.4	db49d009c841f5ca34a888c96511ae936fd9f5533e90d8b2c4d57596f4e5641a
lodash@4.17.21	a9705dfc47c0763380d851ab1801be6f76019f6b67e40e9b873f8b4a0603f7a9
lodash@4.17.21 (src)	4c04561befdf653aef017a42ac5addf68ea943cdfca6bdee5ce04e04e8139f54
lottie-web@5.12.2	a0757321f974527bda3cc2593bf56cc7ffe4578421249ced6ae49ffb1c529f90
luxon@3.4.4	ecd426d1b86f0c92a8b0bf1dfba6604a2d8bc59088700fd30f4f3b18b1013bd3
marked@12.0.1	cb4cf2efefd2b1f602bf2f27d594fc0a26340d2661b60cddfcaac7a7b9261886
mathjs@12.4.0	7228b377458fcdbf53dc905cceb88c2b1da376df4c6bbe76ad50e1c553d29fbe
moment@2.30.1	845c524969edd5b3af9aa6d8718d29fe92e8dbe25b955214a8e064a05a9a5027

Name	SHA256
moment@2.30.1 (src)	7dc0a51c32dae143f2eade235145dfd6a7756388c0f0bf409fa373dd6c233629
monaco-editor@0.47.0	8ea3307c30c3ca6aca9fef3b9e374ab0bf3893e2001724de2a83c3130835ebe2
mustache@4.2.0	d7fd0603512461e8edbd81686bead2ab82df3389b9cca235dd9d5b408848e02a
nanoid@5.0.6	3f64cabce1d66cbaafaf1023070c6672f82b5ac871185186686e7f68541bdbc8
packed_dayjs_evalatob	7206edb49eea5787ab1e3e3eaeced5bcd63d8ef7ab3aa4eac215a35a5437892a
packed_dayjs_evalunescape	a64397cd937bd9234f7f4343913d41eae6d8dd4b023705883fdff7a253e8eac3
packed_dayjs_functionatob	917a8f690223853ffce54cb52a258b4819250de666ee2c939ff4f14b9310c017
packed_mustache_evalatob	721450877b030d440a33100be4cdafacfc08c37cf0b0842322d37bbb980a7b9
packed_mustache_evalunescape	a7115633e834d17d11d7d9f71553d700efe2cb5e8e9693696fedb6f8ea6b9eb7
packed_mustache_functionatob	efeb6fd2153a39b3f98a70c82588ca41815142235e24c464e9ecc10d6e79b773
packed_preact_evalatob	9c0aea604d613a220213bcba5fa1c37b5d4ca28dc4fe76e2050acab04caf2da4
packed_preact_evalunescape	0cfbbb0b31392433276bd4cf281756e2aaca6620322e2ee0ff7580f7e0793cf7
packed_preact_functionatob	87a78a8af98804af50826b85ed0a702624d38ad3833ced7bb75342eca250ae5d
packed_redux_evalatob	f6fa14be07721f8ffe1539c7f56ba2a5ee3aca7904802a6dabaf6408b6dc5d91
packed_redux_evalunescape	89fe6e91ebba2f92beb78da8b8cc011ba19986351acb5109a4e43dcbc27fca1b
packed_redux_functionatob	0b674d205353497c1146461d2064b38f0095517180cd195f6539e6a4891f5987
papaparse@5.4.1	b8e870c5d2b29772f10c9fa9a693c8b896aac8540ed6701e3cc6304c683febdb
pinterest_ct	af4d47bac12442021c6483a1e367902e81de6453414e79834688527de5d3bf

Name	SHA256
pixi.js@7.4.0	2d716f9557afab2966fe17f190d6c2583364d8460b58f488374f4fa5f3770fcb
plausible	cf24eb8d0f972b1cc6e215d1864a52ee29bbf2c08dc91b7d28b8b6a10b0b7005
plotly.js-dist-min@2.30.0	3baddf12a4f47ac22f006078f95c38b21abd53d1511b80e600a60cf0c52bab6e
popper_2_11_8	c212f4b505a86352aed62b24a8f16f999f821ecbe6456c7f3c8a04bc87968782
preact@10.19.3	0151058efb28beb28bade5c6df5718ac9339b04f3afec5350627cdf5f2ef7153
prismjs@1.29.0	11d136a060e78db4a558573a4fdc1a282c89f014c79de9221334d8c971707d0d
quill@1.3.7	c675f57388d3598637c4e0f9fe154bd61dfb1c2086271f944a0bb2b9b059b074
ramda@0.29.1	a3c4757ed608da011c529abb82d88c0323a3750d523e1009d8c6128b5ab3322c
react-dom@18.3.1	35f4f974f4b2bcd44da73963347f8952e341f83909e4498227d4e26b98f66f0d
react@18.3.1	d949f1c3687aedadcedac85261865f29b17cd273997e7f6b2bfc53b2f9d4c4dd
react@18.3.1	28348fef6cb0ed8b2ceeb22deaf824428fd13875d84c73d38f77dd216fc24e7f
redux@4.2.1	54574af2c55419f25dda6b62ba361fc7a4359146ccbb467c3473ca686a0347f7
rxjs@7.8.1	6508e15edacb9eb2c00b8d3fb8fe3022bef1ca1cdb705519524093d4659b3c23
select2@4.1.0-rc.0	f7244fff610595b944f76bf3080d74e3af42b5dd234f8f079e698cc39ac966b0
sentry_7_100	a46617276df65312a984bc4a006f6d275345686f699ff733d10ec70e17328a86
socket.io-client@4.7.4	ad52fc540680945fe7549c0f1b1126b54029dd7eb25f8ce2b079a6242c807011
sweetalert2@11.10.5	fb441ff081cc256b98959da540306b175fb66a28084595c4752bde82d10ba362
swiper@11.0.6	aa8b379f14b0d120d8c9a72effcf7c718ba150d23e37420eaa441c628e42d1ba

Name	SHA256
three@0.160.0	170c6789f43217c96b3170f4b42fafa135de7f7cd48497a4218f9757ee1d49fa
tippy.js@6.3.7	3f0fe70eb26ccf28f6887a192e29d38dd7ef7c2f079a73304ad42ddc7bed37de
toastify-js@1.12.0	42dd6d2bfdd7153d1a702b2b45e468b7c85eec7426bb1e72938397d9a5db396e
underscore@1.13.6	25f436e933246f279adc4967725a4d915e0fc7a6419d3b956a945bb5782dc6e5
underscore@1.13.6 (src)	56bf845439057fbf61e9925905b3c5bea8886604189dcb5312bd5281e4415f5
validator@13.11.0	42758238dc7b954a6923f8ffe9ce4272d8a9490bbbcd71fb7a237d4bf2dccb56
video.js@8.10.0	0957d68a4f0298bad4d37b150b48cbb022f62ac9b660df7056dd6194f69e4dc0
vue@2.7.16	3c1d4b0c549e8de9d4a9bafb12ab70b6a1ac747d07293b98c5b25b6632999afd
vue@3.4.21	4963101441ded7e420c05665e7c616b2f2e3851c99e1cf8af84d29d6f10e77da
vue@3.4.21	269748eb4e30487ac7cd9a3b9f282c4415acaf41b31739ad8fdd6fa9e63433cd
zepto@1.2.0	beb9f5e32ed61fbce010497242a9b6b8219242b5fffc636038e7891510c773725
azure_powershell_analyse_modules	5a528148f7b9961073a4a1173c4c1f8ce647d7b1b95d8776b9d7881d8a604098
azure_powershell_get_azacatwebhook	d9bca300920db76dd0ce534d63d7ede605fec14a37aa3209426821c95bebc0b9
azure_powershell_get_azbotservice	3491689e1615e9ba9149746ef7a01d4b93f6deff91b0818feb817916caa9102b
azure_powershell_get_azconnectednetworkdevice	d3b4cd22b64b91b2dc1efb83a618bef59bfb521030304cd0807b2d8e735940f4
azure_powershell_get_azdeviceregistryschemastry	bf3460589b094a0abdacf5f30ee97192f99e5fcabd356dd65dff888fa45731c7
azure_powershell_get_azevenhub	c397a20132a0a94cfc2e9cfbf6b009893b7a81fdff50f69cdf44a366927d7975
azure_powershell_get_azwvdregistrationinfo	017d7f1a4126cccbf65c04fbed3cb17b4713f8c23bd13d8cef28fb132916c95d

Name	SHA256
azure_powershell_proxycmdletdefinitions	f4866eaa927f24527134d111b0efa04f9d1c4337f2e9825ea48ccf3783843b04
azure_powershell_remove_azcloudhsm	bf27281d8cb76ae54f9e0cf7c83b1210a682f3c0e9a3ff3ef6eb7a1265cd9a31
azure_powershell_remove_azgrafanaintegrationfabric	573177f44eb63d3864de399228b2c1ba018c6aa935808b9df8168ff15200c46b
burnttoast_pwsh32	de3ba23357e183b98d152123c51103223b3d06fe8d854fd4f700f500e9f7515f
chocolatey_install	44e045ed5350758616d664c5af631e7f2cd10165f5bf2bd82cbf3a0bb8f63462
dbatools_convertto_dbatimeline_tests	688b18f27aceef9ef5b93831c23dcd279013df0a624e426f9da4962289b499c3
dbatools_copy_dbaagentproxy	80b3b43cca5323a52268b5107bb3f39133beb1935923734969814f68ea4d4cb9
dbatools_get_dbadbcompatibility_tests	b20c5590791ea2fce0a9eaabc327f0d3ebac208edca9f99936ef245a3913eee5
dbatools_get_dbadbextentdiff	dd041f7fa5bf123029a9a5c279e41fe719613621566179b86b1a8212cc3d5a94
dbatools_get_dbareplarticlecolumn_tests	13089b55254e58faa95747031761e481d44b8be0005f91d0dcc034ddf0031546
dbatools_get_dbargclassifierfunction	9be553d0d8573a90bd832f9e7fe3a3a74e92833147fa911efb91d45a617f1c17
dbatools_get_sqlfilestructure	bfd82e25b65622159c8e28b2c56552c53b96ab987ddb8fe66fabf0df24a9a066
dbatools_new_dbacustomerror_tests	89fe37017367279554a261d33b4245b26fd10fb550b0361227d2b99f8022f638
dbatools_new_dbadbsnapshot	4ed009ae8aefeba30769a18f800f97847f511e39c87725075e31043f8bd763dc
dbatools_set_dbaagentalert_tests	ebff22c3e2324d1a1a2c8325ae50d876e2ad4075f498eeb0362ce2b391c4471a
dbatools_set_dbadbmirror	7bbaf0ca918d37ee4f25ea9e537df4fbd24e76f0ceb215c6e8a3820956076bc2
dbatools_validate_style	c33f9bfa29a1d411f7bd5381d5d40cade439db9bfe40db4b2b9078190d03fb48
dotnet_install	bb1ce92f4397e24d4736a4658b9728fb8f9db64a0d3f8e636ba408a866a6661d

Name	SHA256
importexcel_add_subtotals	012bae9646b50fe4e7ef2ffb67aac0cb6cf571f9c41b9880e6404123703f79c6
importexcel_convertfrom_exceldata	1f5e5992c30e5adc2b6b088ac7b118e880d2a6e21ca608aac9f1a60b081adcc2
importexcel_formatcalculations	abe036dfefb0a3b5d8c5fde90352003e4ccd9c068a0237ef871c23230f43e2775
importexcel_functionalias_tests	82c0413045aeca05c16699c8252a50a309038c4a93cd3b1fe0d8f8e13552765f
importexcel_join_worksheet_sample	2660747a0b679f89a12cda3666dc4b9914811a5023735dd0a5ac10adee1850d7
importexcel_multiplepivottables	740128dac705600ef484c2f308c67b4a39fdead34a3935fc355868ef564b1e5d
importexcel_mutiplevalidations	0f0f0feef27e448bd71e24ae90383ef580e5cdc8605871dd4f3ea2cd5d1c8aa0
importexcel_new_conditionalformattingiconset	87ebf57a255910c77624fdae7408368c04b3d22db9f373e2d6fa01f9e6f320b4
importexcel_tryit	c17a82b95b299556001103053582f1b9606f7dab1cd70fdf4d40b9431ad00c30
install_powershell	e84ea7b8c429c1d474664f5601f23f88201b21152a32f0a47906866da290273c
pester_build	93f950bd2cb9b1ffef561e8d139c477a4794afa22cf8327c2b6aae49e778b003
pester_get_customfailuremessage	2bc6df71e667fdc8ced2f22733f2242e7bc2b2015e5af9dfd90d7b2befefac3e
pester_get_timespanfromstringwithunit	d8f6b7c4d94b7d3a03ffd87a93bfba4f0bfb0e59581a9277ee584527e31ea442
pester_it	189510a89077e2b01e3c740c5ee8e72f777b2114b83c82d34b3c315d086c6077
pester_pester_beforecontainer	670d4ae26cc5ae44c10a19f805c059dcf82e33378768452465626db97a44e273
pester_should_any_tests	a903c87c7f220e253a52432d376e6f39090136b1809acfbbd8804eb307e4e6e9
pester_should_bestring_tests	4b68639cf2d419d81f4e4aeef441ca4ea6b4dbc6aa1c21e25941b6d0ae2546c8
pester_verify_throw	b3932802a40ebdb92fa7a5f2f1343ae844d63c222182cab9dd0664f7aac2805c

Name	SHA256
platyps_build	aa04a54a00922f4a9d72aa10356798a7de320952ddb6cc78ead226516c900c2a
platyps_equatable_tests	6350709b28b7801f01775da8dbc68bc8130fdb6a8c232fb07a62b2bf4024928
platyps_getmarkdownmetadata_tests	759a9ae29988940b04a5df597b4c91351004c89e453fb2a2af89665dee9c89bb
platyps_markdownyaml_tests	d54202ed010b781571151e62abe6c78e10db354ab7ba424fe574d71b6f029399
platyps_scenarios_newmarkdowncomm andhelp_tests	e85ece4bcc33a8a862c66bee1bd0aacb51ffe f27e0166cad6da97eaf23512489
posh_git_get_gitbranch_tests	047c9c0d45a532561ad007bae45f1aabf32d4 ac8529b22dd67d4fb6eb183cfdc
posh_git_gitprompt	0b14df6c9dd11a02e91ea5f60c37a3f379705 1943e99f57acfc5210b607ac2c4
posh_git_gitprompt_tests	8e470991ad864f35b3248fb91d1a19aad3653 2dd906d910a520637df5165fbc5
posh_git_packandlocalinstall	e55a05365f67aea367711ca779f53dd8c1b75 e18f3780bd61aa02e3bd2bdef68
posh_git_profile_example	058d149b9d477a457b4f1fdeefc85ba1e904d f75c8480861ea8ed2d4f6700e79
posh_git_utils	e3fcb6767636a137acb8e47a16f35571c3897 6712879c687a1be622dfcf31817
powershelf_1_build	b6ac98a059bde2a7058da7446d4ea050a2c4e b60700070a04aa590689f2ec5ea
powershelf_add_path_test	d35bf2e74593e0f19c10b044c2df55f13e804 1cbbe2fa31a93d5333500bb75eb
powershelf_invoke_environment	a9b0bd87ee01c9770144b3abc0b178c7df27e 80e2c7f00ae80278b471a8aa09c
powershelf_measure_property_test	180087355c7f27affbe54f1e84892d3fc362d 596f49ce864cff01d8487853a22
powershelf_show_graphqlvoyager	f20398f36526d290e80baaab579592294ada9 32a34ccd70237e9daf45470cf56
powershell_build_debug	c086b252a2625079b381f0c3b3df2e519771a 753ec72cbdf007a9af8403efe95
powershell_build_nightly	ab2b4ce5c9ed665dae5a5fbd6525ae3a69e31 c6af469950801742e9441cc83de

Name	SHA256
powershell_convertto_securestring_tests	11f7489aed42550c206f5ee489c664223536f36a16cbb1255ed5d0167b4041f5
powershell_deactivate_mobilebrowserview	3279eb48adb689bf8080f47872cd5cf8d7bf4c2af287ca60af10436116f7fb1a
powershell_deploy_azureapponly	ebdb1196a8a4941f3a1ab8aef8a91c8b49dd2ac77ce17fc6992cf71e429a8387
powershell_get_cimclass_tests	cae43a71604ecd4ebc05d575e2acc3da56d58bd6def9889028d32dc2f3f8530b
powershell_helpsystem_onlinehelp_tests	60900f0e2ff2ff32a7603ebc0b4fdfe685bb926c93e0cecf90b13cff3ccd4e61
powershell_hierarchical_path_tests	6187d1cbeed61c9e6a68a4a2e48cab8297de53f898f875397704cebfd33361d3
powershell_install_sharepointpowershellhelpermodule	b107a49b760ed33fbb966c33bc2127d97b641c7e72738bfb1f3dadf8d5bc83d3
powershell_new_timespan_tests	5aefd8e816550c7c332c5e351cd792ee19e9c1c1bae5300bba26b73a2734c996
powershell_package_webjobs	5e6bc59207c67190194308d92d75829647c56019470d56a7d2ddc323f7ec3e2c
powershell_parameterbinding_tests	58d369208ed453fc913f1b496f75b61e6b916295852497fb04ef4f9ebf21eb72
powershell_parse_markdownlink	8fd06dd6d652e30810ce71e700e7f25b93d1aad821fdf44c2430426dbe3a4070
powershell_powershellget_tests	a4718277a8c1a18168b79b0b1f74f9024d57525ed162f693b8e324f2585419e2
powershell_provisionmodernpagesandwebparts	924e21e40ed4f850c42d8884e50616d64b1dd0e498799241cefe30a20999f1bc
powershell_unlist_nightly	a421def6798a18ad257de41488b8a346836ae981c4dddf753ba8115fada78a30d
powershellget_findpsresource_tests	9ac6a13b0e51f5027b6483e5cdc8a28874fc82b71702c83aa0bdca9ce8ec6ca0
powershellget_invalidscriptmissingguid	6a992c2202abca2d6ea8f7c9445e905fe749f044e9061ea96c5a2fb1145f6135
powershellget_proxy_tests	334e647b1145bb47eb3ac825fea7eac4b986608df21120af931e53a1185d72b5
powershellget_testpscriptfile_tests	380dfafae455a9bea8f7d6a5a3b609ecfdbcd37c0cef9cef3c94ff7923aacf28

Name	SHA256
powershellget_updatepsscriptfileinfo_tests	9e42cfae7a98dbb53d3b667be30f445a7e97a79d39b9a6b06ad286eeea3993c7
psreadline_build	458bfb4d5b10f30cd51b11d8c2e1ba49e9abebaf5fafb7057ddaac80402f04dd
psreadline_checkhelp	9273fb16119efd62d7296b52052d22179747a5d9c9ea9674d292c910153df013
psreadline_get_keyinfodata	a1dc6bb76e51f2945914a40af00f1895971b8955caf91e89d09416e605b4f0f5
psreadline_psreadline_build	cfd09e21fa171836fc8bbd14ad59ac22ac02d686b7789b5a7a29fc9e876418e6
psreadline_samplepsreadlineprofile	bb7a15b1b4d952544ca209d7ce76e26232242a5e950b88144aba4d28d16d33e8
psscriptanalyzer_avoidmultipletypeattributes_tests	ad7f5b9c92b2dbc6fc066bcb85dc9905894f558e309bcbbac3b6af15a5c57463
psscriptanalyzer_avoidusingdeprecatedmanifestfields_tests	369714cd5f8bf7140069bc7a8b05c34b12bd0655f447aa6ba88f37f412919426
psscriptanalyzer_deadlockregression_tests	4a351d8822e1084737d3f8c6b116e03a53824bcf861db4d317c46a4522475c15
psscriptanalyzer_new_stronglytypedcsfileforresx	9001f25c19614f1741a85d91dcaef0826eb067dc5147f65bfdc30969952fd60d
psscriptanalyzer_pscredentialtype_tests	037bc881cf3f0a8849ef7d21855378af50d325c1d8bdedd17833d2b64c854970
psscriptanalyzer_textedit_tests	e2edb945e85abcc537fcc0b1918267ffd3211a964ea9f0ae2b1e72501da65af9
psscriptanalyzer_usesdscresourcefunctions_tests	acb3ec830ca18ae5b48748a5a0f9213d09b7fdf4eb5eebc988cc9e9850f26ce2
pswritehtml_add_htmlstyle	22a62641ab03b59dba762ab08e8102ab618b15e883e69bcc338ae5e28186e8db
pswritehtml_build_module	03e0466ce44a5aeb49ed05a285eaa7c6bc99ffadf0c538bce186ae5a2c8b49a0
pswritehtml_example7_02	58ff8f3a3ccf9dea9d41eda7f0c50b0409a29df56414022b185c7028620da57d
pswritehtml_example_hierarchicallayout06	54078892106d3807516f18821e835e2f5ac7fa2b3c312143aad79d0ecea40409
pswritehtml_example_tableconditions01	5182ba00b61d2b83eb29a0e3253728878d21e1dce730c6840b39009c14ee50ab

Name	SHA256
pswritehtml_new_chartinternalgrid	163334fcf891e0b2d7eb0b42acc6f07021d8d c68185867ed1ce75497c2e1eb52
pswritehtml_new_htmlmap	f520d82a0cecb70f6478838873cc47418fa3c 9d7931ea62b2b0f0fc497934815
scoop_install	48f6ea398b3a3fa26fae0093d37bd85b13e7e aa5d1d4a3e208408768408e35ae

Appendix B: Malicious Sample Corpus

Omitted due to length – See:

<https://github.com/DSchalla/DeScript/blob/b518f6b8d81de8d36be63b4e4657b28cc49c4f21/corpus/malicious/samples.csv>

Appendix C: Sample faf76865 Call Tree

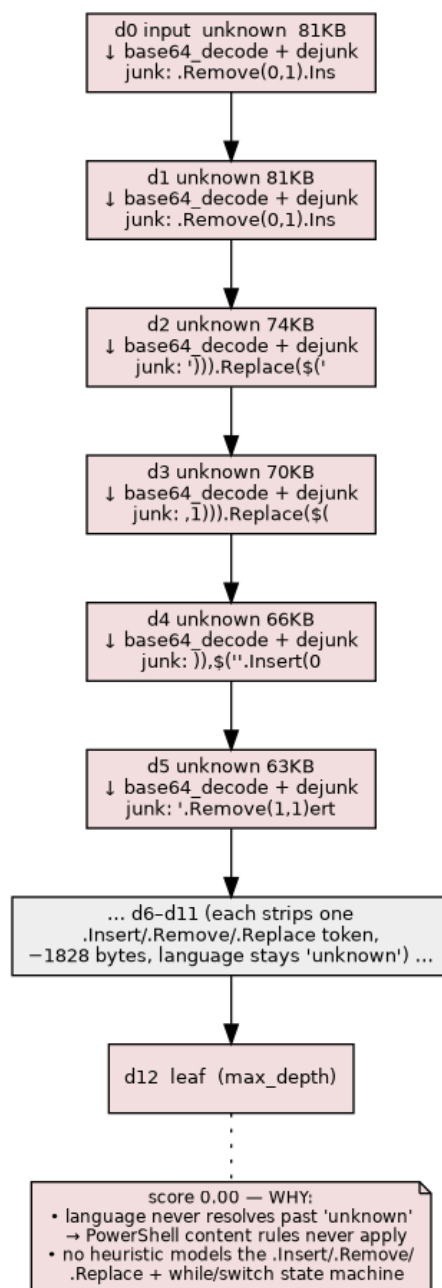


Figure 12: KongTuke Sample

Appendix D: Sample e501319e Call Tree

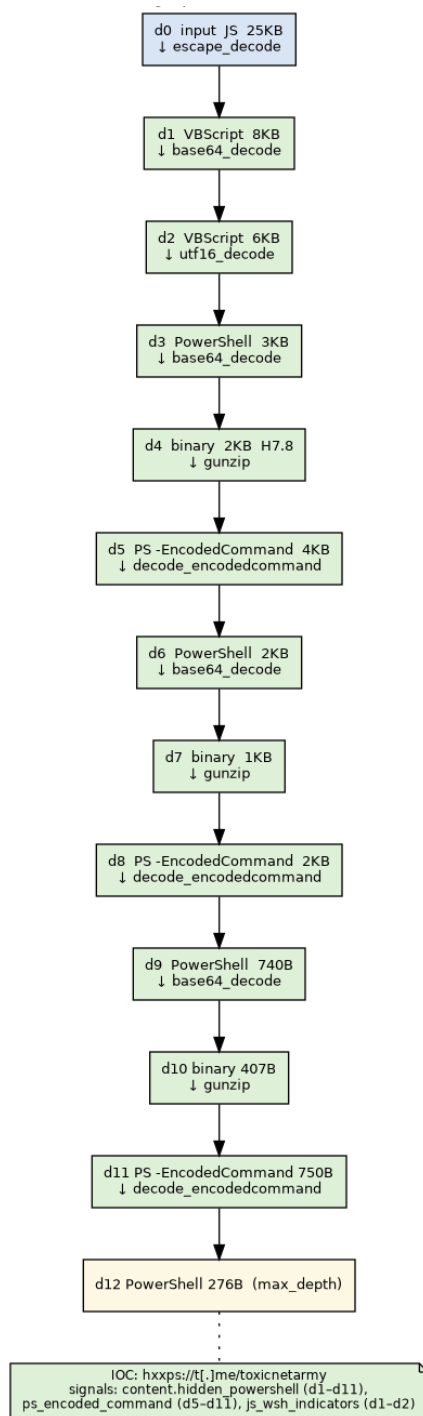


Figure 13: Cross-Language Sample

Appendix E: DeScript Report 47cae3c0

```
DeScript v0.1.0 · 17 heuristics · 16 tools · mode: static
2026-07-03T00:12:31.883367Z [info      ] engine.static_only
excluded_tools=['js.box_js'] run_id=929850e0d909
2026-07-03T00:12:31.883614Z [info      ] analyze.start
run_id=929850e0d909
sha256=47cae3c0b59f0e6a33e7aed184fafd9b978b049a00a88ce1e93ec1ef53
8ffff67 size=42731
2026-07-03T00:12:32.608010Z [info      ] step.ok
child=felf3f96cc94 depth=1 duration_s=0.0 notes=None
parent=11c958b9d59d run_id=929850e0d909
tool=generic.base64_decode
2026-07-03T00:12:32.608091Z [info      ] step.ok
child=1efae3c31d7f depth=1 duration_s=0.0 notes=None
parent=11c958b9d59d run_id=929850e0d909
tool=generic.base64_decode
2026-07-03T00:12:32.611673Z [info      ] step.ok
child=44775219eba6 depth=2 duration_s=0.0 notes=None
parent=felf3f96cc94 run_id=929850e0d909 tool=generic.utf16_decode
2026-07-03T00:12:33.358871Z [info      ] step.ok
child=474c5f801226 depth=3 duration_s=0.0 notes=None
parent=44775219eba6 run_id=929850e0d909
tool=generic.base64_decode
2026-07-03T00:12:33.360440Z [info      ] step.ok
child=c23f2abf30fb depth=4 duration_s=0.0 notes=None
parent=474c5f801226 run_id=929850e0d909 tool=generic.utf16_decode
2026-07-03T00:12:33.365047Z [info      ] analyze.done
iocs=2 label=malicious max_depth=4 nodes=6 run_id=929850e0d909
score=0.8
```

```
┌────────── analysis ───────────┐
| Verdict malicious  score 0.80  |
```

```

|   mode   static                                     |
| layers  6 nodes · depth 4                           |
| tools generic.base64_decode, generic.utf16_decode |
|   iocs   2                                           |
└──────────────────────────────────────────────────┘

```

layers

└─ powershell d0 · 41.7 KB · H5.7 · 47cae3c0b5 →

generic.base64_decode

└─ blob_offset: 3599

└─ decoded_bytes: 10058

└─ junk_token: = Get-Random -M

└─ removed: 116

└─ branches: generic.base64_decode, generic.dejunk

└─ powershell d1 · 9.8 KB · H3.2 · 0d09775dda →

generic.utf16_decode

| └─ encoding: utf-16-le

| └─ decoded_chars: 5029

| └─ powershell d2 · 4.9 KB · H4.4 · 2b4da2a67b →

generic.base64_decode

| └─ blob_offset: 39

| └─ decoded_bytes: 3482

| └─ powershell d3 · 3.4 KB · H3.6 · 742aa68e3d →

generic.utf16_decode

| └─ encoding: utf-16-le

| └─ decoded_chars: 1741

| └─ powershell d4 · 1.7 KB · H5.3 · 0195317b67 →

no_candidate_tool

└─ powershell d1 · 39.9 KB · H5.7 · 4292c2247b →

generic.base64_decode

└─ blob_offset: 3407

└─ decoded_bytes: 10058

indicators (defanged)

kind	value
------	-------

url	
-----	--

hxxp://85[.]239[.]149[.]78:6600/ks3mscqz/dfgsdfgbvbb[.]exe	
--	--

ip	85[.]239[.]149[.]78
----	---------------------

Report + 6 layers written to

out/47cae3c0b59f0e6a33e7aed184fafd9b978b049a00a88ce1e93ec1ef538ff
f67-47cae3c0